

Introduction to containers

How to get software to run reliably when moved from one computing environment to another:

- From a developer's laptop to a test environment,
- From a physical machine to a virtual machine in a cloud.

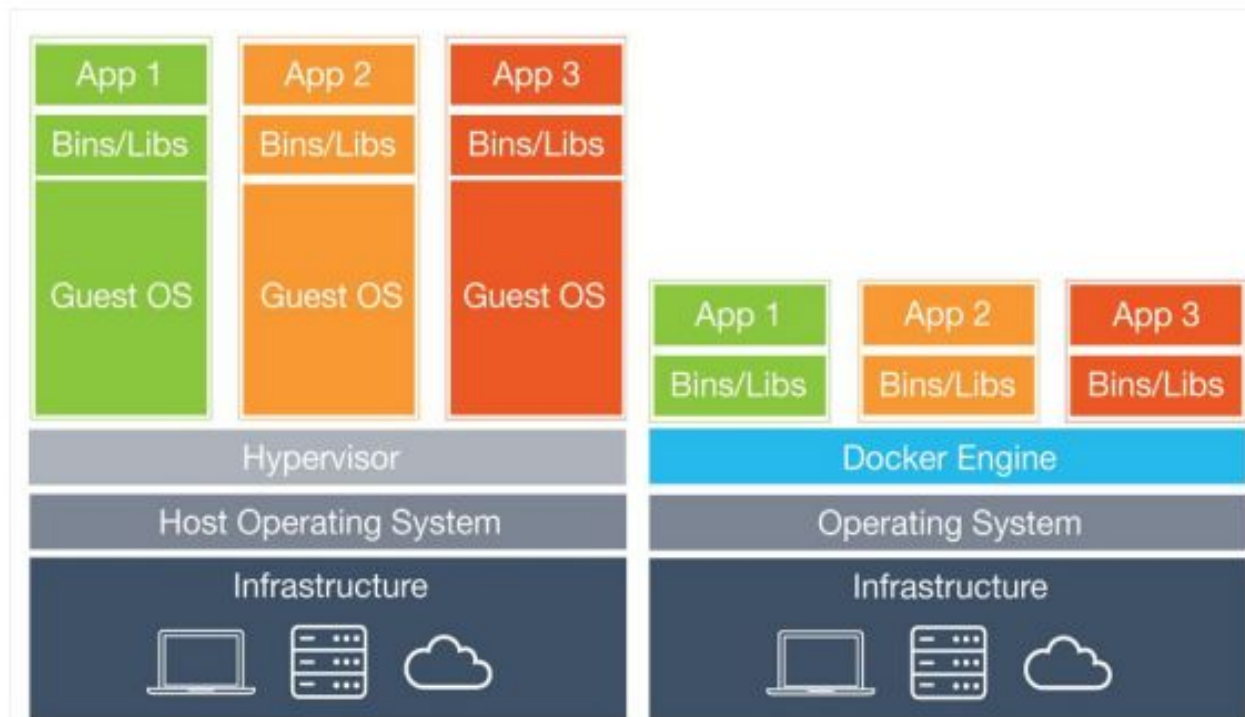
Containers are the solution

- **A container consists of an entire runtime environment: an application, all its dependencies, libraries, binaries, configuration files ... bundled into one package.**
- **Containers encapsulate components of application logic. These components are provisioned only with the minimal resources needed to do their job.**
- **Unlike virtual machines (VM), containers have no need for embedded operating systems (OS); calls are made for OS resources via an application programming interface (API)**

Containers are the solution

Containers

- Isolated environments to run applications/services
- Images include all software dependencies
- Prescriptive, portable, easy to build, quick to deploy



Virtualization vs. “containerization”

- A virtual machine includes an entire operating system + application.
- A physical server running three virtual machines would have:
 - a hypervisor,
 - three separate operating systems running on top of it.
- By contrast a server running three containerized applications runs a single operating system
- Each container shares the operating system kernel with the other containers.
- That means the containers are much more lightweight and use far fewer resources than virtual machines.

Virtualization vs. “containerization”

- A container may be only tens of megabytes in size
- A virtual machine with its own entire operating system may be several gigabytes in size.
- A single server can host far more containers than virtual machines.
- Virtual machines may take several minutes to boot up their operating systems and begin running the applications they host
- Containerized applications can be started almost instantly.

Benefits of containers

- **Lightweight:** Containers share the machine OS kernel, eliminating the need for a full OS instance per application and making container files small and easy on resources.
- **Portable and platform-independent:** Containers carry all their dependencies with them, meaning that software can be written once and then run without needing to be re-configured across laptops, cloud, and on-premises computing environments.
- **Supports modern development and architecture:** Due to a combination of their deployment portability/consistency across platforms and their small size, containers are an ideal fit for modern development and application patterns—such as DevOps, serverless, and microservices
- **Improves utilization:** Like VMs before them, containers enable developers and operators to improve CPU and memory utilization of physical machines.

What about security?

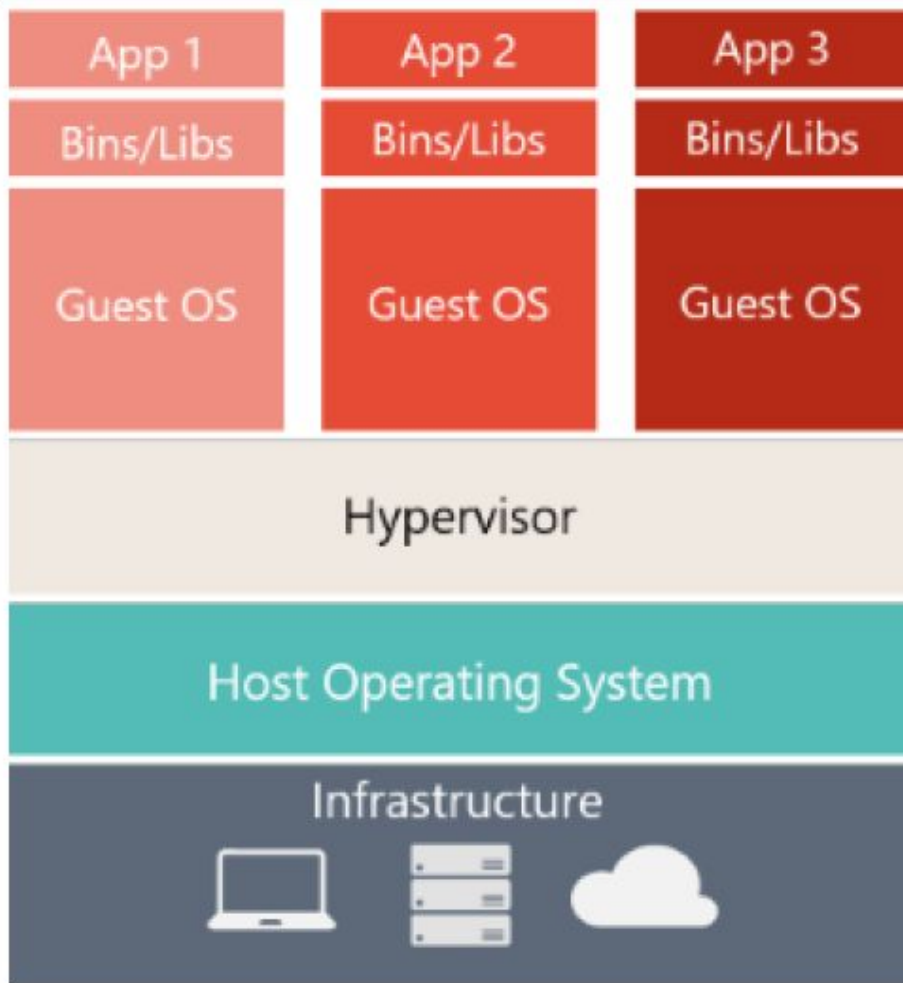
- Container is a method that isolates processes and
- resources ... But ...
- Containers are not as secure as virtual machines
- if there's a vulnerability in the kernel, it could provide a way into the containers that are sharing it
- That's also true with a hypervisor, but since a hypervisor provides far less functionality than the whole OS, it presents a much smaller attack surface.

Docker

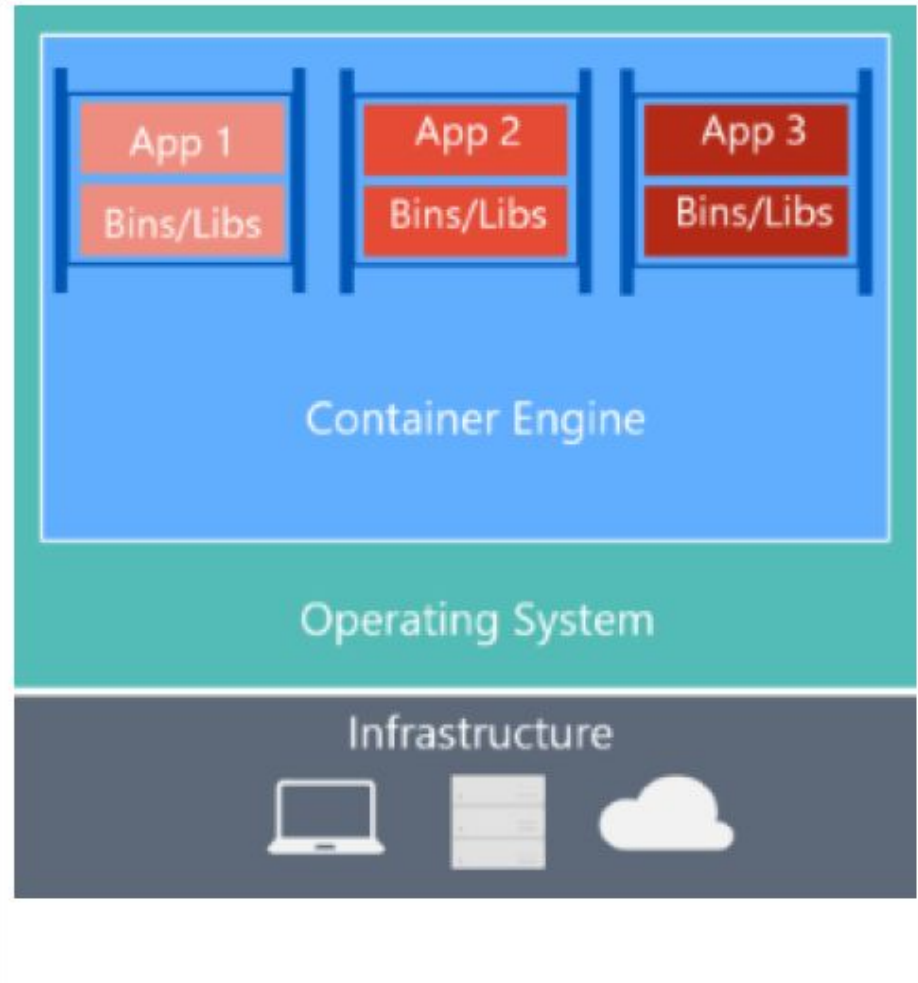
- **Extremely popular container implementation**
- **Docker is an open-source project for automating the deployment of applications as portable, self-sufficient containers that can run on the cloud or on-premises.**
- **Docker is also a company that promotes and evolves this technology, working in collaboration with cloud, Linux, and Windows vendors, including Microsoft.**

Comparing Docker containers with virtual machines

Virtual Machines



Docker Containers



Comparing Docker containers with virtual machines

Virtual Machines	Docker Containers
Virtual machines include the application, the required libraries or binaries, and a full guest operating system. Full virtualization requires more resources than containerization.	Containers include the application and all its dependencies. However, they share the OS kernel with other containers, running as isolated processes in user space on the host operating system. (Except in Hyper-V containers, where each container runs inside of a special virtual machine per container.)